National University of Computing and Emerging Sciences, Lahore

	Course	Data Structures	Code	CS2001
	Program	BS(CS)	Semester	Fall 2026
	Due Date	As announced	Marks	100
	Type	Assignment 1	Page(s)	3
Note	*Plagiarized or copied submissions will be dealt with according to course policy.*			
	Course instructor: Rana Waqas Ali		Teacher Assistant: Syed Aoun Haider Sherazi	

Important Instructions:

1. Submit one C++ source file named <RollNo>_BST.cpp and your short written answer for Part e.
2. Use your own pointer-based BST. STL maps/sets, AVL trees and rotations are not allowed.
3. All parts must be attempted. Handle invalid inputs and unsuccessful operations properly.
4. Work individually. Discuss any confusion with the course instructor or TA.

Question 1: Campus Shuttle Dispatch Scheduler [100 marks]

A university operates a single shuttle departure bay. During the day, the transport office schedules new trips, cancels departures and changes departure times. The staff also need to find the next available shuttle and prepare lists of trips within a given time period. To manage these requests, the office requires a small C++ application.

Develop a class ShuttleBST using an ordinary (unbalanced) binary search tree. Each node will store a shuttle departure, with departureTime as its key. Use the following information for each record:

departureTime	Integer minutes after midnight; e.g., 540 represents 09:00. It is the unique BST key.
destination	Single-word destination (HOSTEL, LIBRARY, GATE, etc.).
seatsAvailable	Number of seats remaining for that trip.

Only one shuttle may depart at a given minute. Departure times must be in the range 0 to 1439, and a newly scheduled trip must have a positive number of seats. No balancing is to be performed.

Part a: [40 marks]

1. **Node and class [8 marks]:** Create the node structure containing departureTime, destination, seatsAvailable, and left/right pointers. Define ShuttleBST with a root pointer, a constructor, and a destructor to free the tree.
2. **scheduleDeparture(time, destination, seats) [10 marks]:** Insert a new departure at its proper position. Reject duplicate departure times and invalid details.
3. **findDeparture(time) [7 marks]:** Search for the given departure time and display the full record. Display a suitable message if it does not exist.
4. **cancelDeparture(time) [15 marks]:** Remove a scheduled departure. Your function must handle a leaf node, a node with one child, a node with two children and deletion of the root. An unknown departure time should leave the tree unchanged.

Part b: [35 marks]

The transport office also requires the following operations. These must be performed on the same BST without reconstructing the entire tree.

1. **rescheduleDeparture(oldTime, newTime) [15 marks]:** Change the departure time of an existing shuttle while keeping its destination and remaining seats. The new time must be valid and unoccupied. If the request cannot be completed, the original record must remain unchanged. If both times are equal and the trip exists, treat the request as successful without making a change.
2. **nextDepartureAfter(time) [10 marks]:** Find the trip with the smallest departure time strictly greater than the given time. If there is none, display "No upcoming shuttle." Use BST traversal logic rather than copying and sorting the departures.
3. **listDeparturesBetween(start, end) [10 marks]:** Print all departures falling within the inclusive interval [start, end] in increasing time order. Avoid visiting subtrees that cannot contain a valid result. Handle an empty result and a reversed interval (start > end).

Part c: [10 marks]

1. **boardPassengers(time, count) [5 marks]:** Deduct count from the remaining seats of the corresponding shuttle only if the trip exists, count is positive, and sufficient seats are available. Otherwise, reject the request without altering the record.
2. **printDailySchedule() [5 marks]:** Display all scheduled shuttles in increasing order of departure time, including the destination and available seats. Handle an empty tree.

Part d: [10 marks]

Write a menu-driven main() function through which the transport office can use all the operations. The menu may follow the choices below (you may change the wording of the messages):

1, 2, 3	Schedule / Search / Cancel a departure
4, 5, 6	Reschedule / Next departure / Departures within a time range
7, 8, 0	Board passengers / Print full schedule / Exit

All unsuccessful operations must leave the tree and its stored records unchanged. The BST must remain the primary storage structure; do not maintain an additional array, vector, map or set containing copies of the departures.

Part e: [5 marks]

Insert the following departure times into an initially empty BST in the given order: 540, 480, 660, 510, 600, 720, 570.

1. **[2 marks]:** Draw the resulting BST.
2. **[1 mark]:** Write the inorder traversal.
3. **[1 mark]:** State the result of nextDepartureAfter(600) and give a short explanation.
4. **[1 mark]:** State the worst-case time complexity of searching an unbalanced BST.

Sample run

The following sequence illustrates the required behavior. The commands are shown for explanation; your menu does not need to use these exact words. Start with an empty tree.

Command	Expected result
SCHEDULE 540 LIBRARY 18	Departure for 09:00 is inserted.
SCHEDULE 660 GATE 12	Departure for 11:00 is inserted.
SCHEDULE 600 HOSTEL 10	Departure for 10:00 is inserted.
SCHEDULE 600 STADIUM 20	Rejected because a trip is already scheduled at 600.
NEXT 570	Returns 600 HOSTEL 10.
BOARD 600 4	Remaining seats for time 600 become 6.
MOVE 660 615	GATE moves to time 615; time 660 is removed.
RANGE 540 615	Displays 540 LIBRARY 18, 600 HOSTEL 6, and 615 GATE 12 in this order.
CANCEL 540	Removes the departure scheduled at 540.

Points to check before submission

- Search, deletion, next departure and range queries on an empty tree.
- Duplicate keys, invalid departure times and non-positive initial seats.
- Deletion of a leaf, a single-child node, a two-child node and the root.
- Rescheduling to an occupied time, a missing source, or the same time.
- Last possible departure, reversed/empty ranges, and ordered output.
- Boarding zero passengers, too many passengers, or passengers on a missing trip.

Submission

Submit <RollNo>_BST.cpp and a brief PDF or image containing the hand-drawn/written solution of Part e, through the channel announced by the instructor. Your code should be readable and properly divided into functions. Test all operations before submission.

Marks distribution

Part a - BST implementation: 40 marks
Part b - Scheduling operations and queries: 35 marks
Part c - Passenger handling and schedule: 10 marks
Part d - Menu and validation: 10 marks
Part e - BST trace and analysis: 5 marks